SIGCOMM 2026 Paper #366 Reviews and Comments
===
Paper #366 Scalable DNS Configuration Verification via the LEC-native Query Resolution Graph

Review #366A
===

Overall merit
----
3. Weak accept (A paper that you are okay with being accepted, you will not argue against it. The paper is correct with potentially limited novelty or unconvincing evaluation.)

Experimental methodology
----
4. Good

Reviewer expertise
----
3. Knowledgeable - Have published or have worked in this area or a related area

Paper summary

This paper proposes a DNS verification approach that speeds up verification time compared to GRoot. My understanding of the core difference is that GRoot takes record names across all zone files and name servers constructs equivalence classes from that. Aether instead constructs them locally for each zone file. The evaluation shows that Aether creates 10x fewer ECs, and can improve verification time from 65 milliseconds to 12 milliseconds.

Reasons to accept

- The core idea behind improvement (M + N instead of M * N) is reasonable.

Reasons not to accept

- Some of the technical content is imprecise and inconsistent. They are not all writing issues in my opinion. I have provided examples in "Comments for authors".

- At the end of the day, the actual performance improvement does not seem necessary. For a data set of over a million zone files, GRoot can still finish verification in 65 milliseconds at 95-percentile. While reducing it to 12 milliseconds is a ~5x reduction, it is shaving a few milliseconds.

Comments for authors

I was intrigued by the paper's idea, and I appreciated the clear examples at the beginning of the paper. However, as I went on, there were some places that, in my understanding of the paper, seemed imprecise and inconsistent. I'll list them below.

Equivalence class description and arguments. The earlier parts of the paper imply that one of the new ideas compared to GRoot is that GRoot reasons at per-record domain-name granularity, and as such, fails to merge records that have identical actions. However, when we get to section 3.2 and Algorithm 2, my understanding is Aether does the exact same thing, but per zone. Quoting from the paper: "First, we aggregate records with the **same** $rname$ and $rtype$". Algorithm 2 starts from these records, gets the "space" for each one, and each becomes a match-action rule. The space seems to be every record with that prefix. Later on, in 4.2, the paper mentions that "LECs generated by Section 3.2 can back-trace to a specific record, allowing us to filter LECs related to the query through $rname$.". To me, this confirms my interpretation that the LECs in Aether are also per-record, and there is no merging of records, unlike what the paper suggests earlier.

Where I think the reduction in ECs is coming from is that by constructing ECs globally, GRoot may create ECs that will not actually occur. For example, in Figure 1, because there is "went.com" and "went.net", the label tree that GRoot constructs has net and com as parents of went. So, if we add a child "x" under went, we are adding to ECs "x.went.com" and "x.went.net", independent of whether they occur. This happens in the example in the figure. "where.went.net" does not exist in any zone file yet there is a path for it in GRoot's label graph, and it will turn into an EC. This does not

come up in the paper, and was a source of hesitation for me regarding this paper.

Other. These are more minor but at the end of section 3's preamble, the paper mentions: "We compute LECs at the nameserver level because zonefiles on the same nameserver may cover overlapping or nested domain spaces (e.g., a.com and b.a.com). Generating LECs per zonefile would therefore result in overlapping classes.". In 3.2, the paper mentions that Aether uses the origin of a zone files to create disjoin guards, and then computes LECs for each zone. Moreover, multiple times, the paper correlates having fewer ECs with lower verification time. That is not fully accurate. It depends on what operations are done per-EC. You can have fewer ECs but do more for each EC, in which case, it is generally not clear which approach would do better overall. I understand that the evaluation results show that Aether does better overall, but attributing it to just fewer ECs does not seem fully accurate to me.

Performance improvement At the end of the day, the actual performance improvement does not seem necessary. For a data set of over a million zone files, GRoot can still finish verification in 65 milliseconds at 95-percentile. While reducing it to 12 milliseconds is a ~5x reduction, it is shaving a few milliseconds.

Review #366B
===

Overall merit
----
3. Weak accept (A paper that you are okay with being accepted, you will not
   argue against it. The paper is correct with potentially limited novelty
   or unconvincing evaluation.)

Experimental methodology
----
4. Good

Reviewer expertise
----
1. None - Have little or no prior knowledge in the area

Paper summary

This paper studies automated analysis of DNS to detect configuration errors

Reasons to accept

This paper offers an elegant framing for thinking about DNS configuration and applies the idea to a real dataset to find some misconfiguration errors

Reasons not to accept

Somewhat niche topic

Unclear how impactful the bugs found are

Unclear how many would be found with prior techniques

Comments for authors

I found this paper to be easy to read and the methodology is intuitive and promising in terms of efficiency and simplicity. It would help me if the paper successfully showed:

* that the issues found are substantitve. Are they for domains/subdomains in active use? I wonder if they're just little used/forgotten about domains

* There is no comparison to prior work. How many of these errors would be found with prior techniques? What would be missed?

I also would like to see the techniques extended to DNSSEC (which was not mentioned in the paper but poses a lot of additional opportunities for configuration errors)

Review #366C
==========

Overall merit
---
2. Weak reject (A paper that you want to reject, but will not champion rejection if there are positive reviews (e.g., '3's or above). The paper is correct with potentially limited novelty or unconvincing evaluation.)

Experimental methodology
---
2. Poor

Reviewer expertise
---
2. Some - Have some understanding of the area or have reviewed papers in the area before

Paper summary

DNS configuration is incredibly complex and prone to misconfigurations that can lead to massive outages. Previous efforts to statically verify DNS configurations (like GROOT) suffer from severe scalability issues and lack incremental update support because they rely on globally refined equivalence classes (GECs).

This paper introduces AETHER, which maps authoritative DNS resolution to a data-plane match-action pipeline. By aggregating the query space into Local Equivalence Classes (LECs) per nameserver and using a BDD-based representation for variable-length domain names, AETHER effectively mitigates the state explosion problem; the paper claims impressive results: up to 255.7x speedup for end-to-end verification and up to 3370x speedup for incremental updates.

However, I think that beneath these impressive performance numbers lie several fundamental compromises in the verification model and a concerning evaluation methodology.

Reasons to accept

+ Tackles a critical problem: DNS configuration verification at scale.

+ The conceptual shift from global ECs to local ECs (LECs) by treating DNS resolution as a match-action data plane pipeline is insightful and highly effective for incremental updates.

+ Clever use of bounded BDDs to handle variable-length string matching in DNS names.

Reasons not to accept

- The evaluation dataset is cherry-picked; the authors dropped over 180,000 zones simply because they failed a syntax check (named-checkzone), explicitly avoiding the edge cases a verification tool is supposed to handle.

- The model oversimplifies reality to achieve its speedups, completely excluding DNSSEC and caching, which are the sources of many real-world DNS failures.

- It relies on heuristics (a fuel parameter k and artificial label padding $n+\delta$) to prevent infinite loops, making this bounded model checking rather than complete formal verification.

Comments for authors

To be very clear: I really like the core idea of this work. Mapping DNS resolution to a data-plane match-action pipeline and using LECs to achieve scalability and incremental verification is a fantastic approach.

My main concern with accepting it at this time is that the impressive scalability numbers seem to come at the heavy cost of model fidelity and empirical rigor. There are several methodological choices that undermine the claim of "complete" verification, leading me to wonder if the system can actually handle the messy reality of the Internet.

Dataset Filtering and "Invalid" Zones

My biggest concern is the evaluation methodology in Section 6.2. You mention that out of 1.36 million zones, you filtered out roughly 180,000 zones because they violate the named-checkzone directive (e.g., quotes not enclosed).

This is highly problematic. The entire purpose of a configuration verification tool is to catch errors that humans make. By throwing away 13% of the dataset because it contains syntax errors or edge cases, you are artificially sanitizing the workload. How does AETHER's BDD construction and symbolic execution perform when faced with these malformed, real-world edge cases? A robust verifier must be able to gracefully handle or properly categorize these inputs, not simply drop them to improve evaluation times.

True Verification vs. Bounded Model Checking

To handle DNAME rewrites and potential infinite loops, the paper introduces a label bound $n+\delta$ and a "fuel parameter" k to limit recursion depth.

While this prevents the verifier from crashing, it means AETHER is performing bounded model checking, not true verification. If an adversarial or accidental configuration creates a CNAME chain that requires $k+1$ hops, AETHER will presumably terminate early and miss the behavior. The paper needs to be much more upfront about this limitation and provide an analysis on how to safely determine k and δ for production networks.

The Cost of an Oversimplified Model

In Section 2.1, you explicitly exclude DNS caching and DNSSEC to "keep the model focused and tractable". However, as seen in many recent major outages, the interaction between authoritative server responses, resolver caching (TTL mismatches), and DNSSEC validation failures is where the most devastating bugs hide.

While I understand the need to scope the paper, treating DNS solely as a stateless forwarding plane ignores the stateful properties (caching, crypto validation) that make DNS fundamentally different from IP forwarding. The paper would be much stronger if it included at least a discussion on how the LEC-native graph could be extended to model these critical components in the future.

Review #366D
==========

Overall merit
---
2. Weak reject (A paper that you want to reject, but will not champion rejection if there are positive reviews (e.g., '3's or above). The paper is correct with potentially limited novelty or unconvincing evaluation.)

Experimental methodology
---
3. Average

Reviewer expertise
---
2. Some - Have some understanding of the area or have reviewed papers in the area before

Paper summary

This paper presents AETHER, a scalable and incremental DNS configuration verification framework. The authors observe that existing tools like GROOT suffer from limited scalability on large datasets and lack support for incremental updates because they rely on fine-grained, global equivalence classes (GECs).

To address these limitations, AETHER adopts a Local Equivalence Class (LEC)-native approach. Instead of using the domain-name granularity found in GROOT, AETHER groups domain names with identical match-action behaviors at each local nameserver into LECs. This enables AETHER to directly reason on a compact DNS resolution graph, where nodes represent the logical input and output ports of nameservers. AETHER employs symbolic query execution to traverse this resolution

graph to generate traces and check for properties.
Furthermore, AETHER introduces an incremental verification method that restricts recomputation to only the affected graph regions during configuration changes.

Evaluation on real-world datasets shows that AETHER achieves up to 255.7× speedup for end-to-end verification and up to 3,370× speedup for incremental updates compared to GROOT.

Reasons to accept

- The evaluation demonstrates substantial performance gains over GROOT, achieving speedups of two to three orders of magnitude in both full and incremental verification scenarios.

- The paper provides an interesting insight by leveraging established data-plane verification (DPV) techniques to address and improve DNS configuration verification.

Reasons not to accept

- While the paper compares extensively against GROOT [SIGCOMM '20], it lacks a comparison with more recent advancements mentioned in the related work, such as VeriDNS [10] and Octopus [48].

- While DNS resolution graph is novel for DNS verification, the rest two designs, symbolic execution and incremental verification, are widely deployed in the network verification literature. The paper does not sufficiently distinguish its specific contributions in these components from existing DPV or prior DNS verification methods.

Comments for authors

Thank you for submitting to SIGCOMM. This paper addresses an important and practical problem, and the reported performance gains are promising. I also find the LEC-native DNS resolution graph to be an interesting perspective. However, I am not fully convinced that the current version is strong enough for acceptance. In particular, the paper does not yet clearly distinguish its novelty from prior DNS and data-plane verification systems, and the evaluation leaves some important gaps in system positioning and empirical support. Overall, I view this as a promising piece of work, but not yet sufficiently mature for acceptance in its current form. I have some comments for authors:

- In the evaluation, the authors attribute the maximum speedups of AETHER to its parallel computation during the generation of local equivalence classes, whereas GROOT requires all zonefile equivalence classes to be computed sequentially. However, this parallel design is not explicitly detailed in the Design sections (Sections 3–5). It is unclear how much of the performance gain stems from the designs in this paper versus implementation-level optimizations. A breakdown of these factors would make the results more convincing.

- In the introduction, the authors mention that the symbolic execution algorithm of AETHER introduces visited-state cache to prune rewrite-induced cycles. However, this mechanism appears to be missing from the detailed design sections.

- Incremental verification has been studied in DPV. The authors should clarify the unique challenges of implementing incremental verification in the DNS context compared to packet forwarding.

Given that VeriDNS[10] also supports the incremental DNS verification, it could better position this work if the authors could clarify why VeriDNS is considered "approximate" and provide a more explicit comparison of how AETHER's "complete" incremental check avoids the weakness of VeriDNS.

- Beyond maximum and median speedups, please provide the geometric mean or average speedup. The CDFs in Figure 4 suggest that more than $100\times$ speedups of AETHER might be extreme outliers.

- Since the performance gap between AETHER and GROOT varies significantly across different DNS cases, it would be insightful to include an analysis of these variations, particularly identifying the specific zone characteristics or configuration patterns where AETHER's LEC-native approach yields the most substantial benefits.

Review #366E
==========

Overall merit

---
2. Weak reject (A paper that you want to reject, but will not champion
   rejection if there are positive reviews (e.g., '3's or above). The paper
   is correct with potentially limited novelty or unconvincing evaluation.)

Experimental methodology
---
3. Average

Reviewer expertise
---
2. Some - Have some understanding of the area or have reviewed papers in
   the area before

Paper summary

This paper proposes a DNS verification tool that is faster than GRoot, leveraging a graph-based technique and symbolic execution. The evaluation shows that it consistently and significantly outperforms GRoot.

Reasons to accept
---
1. Addresses an important problem
2. Demonstrates significant improvement over GRoot

Reasons not to accept
---
1. Only compares against GRoot
2. Evaluation of error detection is conducted on a relatively small dataset

Comments for authors

Although this paper is outside my area of expertise, I generally enjoyed reading it. DNS resolution is a practical and important problem, and large-scale outages are often rooted in DNS misconfigurations. The proposed solution is intuitive, based on a reasonable insight: localizing recomputation to only the affected records or routes. The evaluation results are promising, showing substantial improvements over GRoot.

That said, I have a few concerns. First, the paper frames its contributions primarily in comparison to GRoot and only evaluates against it. While reading, I initially assumed that AETHER was the first solution for incremental DNS verification, which seemed surprising. Later, the related work section mentions that VeriDNS also supports incremental updates. This raises the question of whether VeriDNS is a more appropriate baseline than GRoot. Simply stating that “AETHER supports complete incremental verification, unlike VeriDNS” is insufficient to clearly distinguish their capabilities and performance. Since incremental verification is the main contribution of this paper, a more thorough comparison with VeriDNS is necessary. While the improvement in construction time over GRoot is impressive, construction appears to be more of a one-time cost, which may reduce the practical significance of this advantage.

I also appreciate that the evaluation covers both error detection and performance. However, the error-finding experiments are conducted on a relatively small campus dataset. It would strengthen the paper to include evaluations on larger datasets, particularly in incremental settings.

Comment @A1 by Reviewer A

Dear authors, please find the rebuttal questions below:

1. Clear clarification of the main differences with GRoot. Specifically, what are the main differences in the EC generation process between AETHER and GRoot that contribute the most to the performance improvements? In what kinds of zone files/configurations would this improvement be the most pronounced? (Reviewers A and D)

2. How can AETHER's approach be extended to support the more stateful components of DNS such as DNSSEC and caching? (Reviewers B and C)

3. Are the reported performance improvements meaningful? Please report other metrics such as mean (Reviewer D) and describe scenarios where reducing verification time by ~10s of milliseconds will

impact the usability of verification in practice (Reviewer A).

4. Clarify/justify evaluation choices (Reviewers B, C, and E) . Specifically:
   - Why does the data set remain representative after removing the 180K with syntax problems?
   - Is the campus data set used for demonstrating the bug finding capabilities on par with what is used in the literature and is available publicly?
   - Is the set of bugs found on par with the literature in their significance and number?

5. Please describe the differences with the existing incremental DNS verification tool, VeriDNS [10], in more detail (Reviewers D and E)

Rebuttal Response by Author [Yao Wang <yao.wang.xmu@gmail.com>] (400 words)

A1. The primary difference driving AETHER’s performance improvement is its resolution(action)-aware aggregation. While GRoot generates global ECs strictly per-record—causing an explosion of ECs and redundant local resolutions during verification—AETHER constructs Local ECs and a query resolution graph (QRG). Utilizing space-based queries and BDD-based encoding (§4.2), AETHER drastically reduces EC counts, repeated resolutions, and per-LEC computation costs. This advantage will be more pronounced in configurations with dense records sharing the same actions, as AETHER completely eliminates GRoot's redundant per-record verification overhead.

A2. AETHER can natively incorporate DNS caches. Assuming the cache state can be collected and the querying logic between resolution and authoritative servers is defined, cached entries are simply treated as dynamic, time-bound DNS records. These records can be directly mapped into AETHER’s QRG. AETHER’s incremental verification perfectly aligns with the need to efficiently process the continuous updates of DNS caches without incurring the overhead of global re-verification

For DNSSEC’s "chain of trust," resolution dynamically generates dependent sub-queries (e.g., fetching DS/DNSKEY). Since AETHER uses path-based QRG verification, we can expand graph semantics to model these multi-step dependencies, treating DNSSEC signature validations as specific match conditions within AETHER's match-action tables.

A3. Mean speedups mirror our median results: ~5× (full) and ~50× (incremental). A 50× incremental speedup is critical for ensuring strictly real-time reactions in high-frequency CI/CD deployment pipelines.

Additionally, our evaluated dataset averages only ~20 records/zone. Because AETHER scales proportionally with LECs rather than raw records, current ms-level savings will magnify into seconds or minutes on massive real-world enterprise zones where per-record approaches like GRoot would bottleneck.

A4. We used named-checkzone to filter syntax-error and RFC-violating records (e.g., conflicting CNAME/A records for the same rname) because our engine cannot model invalid configurations, not to cherry-pick. The removed records will be open-sourced upon publication.

Due to privacy concerns, the campus dataset currently cannot be made public. However, it comes from real-world deployments, and all discovered bugs have been acknowledged. AETHER successfully identifies all bug types (shown in Table 1) reported by GRoot, [29], and RFC 8484.

A5. VeriDNS essentially trades semantic fidelity for scalability: its approximate state graph is limited to syntax-level detection and misses semantic errors stemming from dynamic or temporal interactions. In stark contrast, AETHER guarantees exact verification. By combining an LEC-native architecture with precise BDD-based encoding and symbolic execution, AETHER mathematically compresses the state space without any loss of fidelity, delivering both uncompromising accuracy and highly efficient incremental verification.

Comment @A2 by Reviewer A

Dear authors,

Thank you for your rebuttal response. Unfortunately, there was not sufficient support from the reviewers for the paper in the online discussion, and there were some concerns about prior work such as GRoot and VeriDNS, and ultimately, the decision was to reject the paper. We hope the reviews provide meaningful feedback for future revisions of the paper.